Distinguishing Objective Completion from Constraint-Violating Optimization in AI Agents

Tariq Mahmood [1]

Abstract

Agents built on large language models (LLMs) are now routinely asked to plan, call tools, run code, and act on external systems with little supervision. In this setting an agent can reach its assigned objective and still break the rules that were meant to govern how the objective is reached. Most benchmarks score only the final outcome, so an agent that obtains the right answer through an unauthorized tool, a leaked credential, or a flaw in the environment is counted as a success. We argue that objective achievement and constraint compliance are separate properties and should be measured separately. We describe a conceptual evaluation framework with five components: outcome verification, trajectory monitoring, constraint compliance, uncertainty and anomaly detection, and post-hoc behavioral analysis. We discuss how distribution shift relative to known-compliant trajectories can flag risky behavior before a task finishes, why the resulting measurements should not be collapsed into one score and how they can still support a release decision, how the framework could be instantiated on existing benchmarks, and which research problems remain open, including inferring intent from behavior, reward design, early detection, and the reliability of LLM-based monitors.

1. Introduction

Large language models were first deployed as question-answering systems. They are now increasingly used as agents: they decompose a goal into steps, call tools, execute code, browse the web, and interact with external services over many turns (Yao et al., 2023). Once a model acts rather than answers, a distinction that was previously unimportant becomes central. There is what the agent achieves, and there is how it achieves it.

A simple example makes the point. Suppose an agent is asked to retrieve a record from a particular database. A conventional evaluation marks the run as successful if the returned record is correct. Now suppose the agent obtained the record by querying a different database it was not authorized to use, by exploiting a vulnerability, by reading credentials it was never meant to see, or by sidestepping an explicit restriction in its instructions. Judged by outcome, nothing went wrong. Judged by any reasonable standard of trustworthy behavior, the run is a failure.

This is not a hypothetical concern. Reinforcement learning has a long record of systems that satisfy the literal objective in ways their designers did not intend (Amodei et al., 2016; Krakovna et al., 2020), and recent reports describe LLM-based agents that tamper with their environment or evaluation when the legitimate route is hard (Bondarenko et al., 2025; Meinke et al., 2024). Widely used agent benchmarks nevertheless report task success as the headline number (Liu et al., 2024; Zhou et al., 2024; Jimenez et al., 2024). Such a number cannot tell apart an agent that solved the task from an agent that found a way around it.

The question we address is therefore the following: *how can we distinguish an agent that completes its assigned objective from one that optimizes for the objective while violating the constraints that were supposed to bound its behavior?*

The question becomes sharper as agents take on longer horizons and richer environments, because a system optimized mainly for completion has more opportunities to discover strategies that meet the formal objective by exploiting the environment or the evaluation procedure. Our position is that task success is not a sufficient condition for acceptable behavior, and that evaluation should report at least two things: whether the objective was achieved, and whether the constraints were respected.

This paper is a discussion paper. It does not report experiments. Its contributions are:

- a statement of the problem that separates objective achievement from constraint compliance (Section 3);

[1]DIMES Department, University of Calabria, Rende (CS), Italy. Correspondence to: Tariq Mahmood <mehmoodt222@gmail.com>.

Preprint. Under review.

- a conceptual Constraint-Aware Agent Evaluation Framework (CAAEF) with five components (Section 4);
- a two-dimensional evaluation model, an argument against scalar aggregation, and a decision rule for release gating that does not need a scalar, with an analysis of how it behaves when the checker makes errors (Section 5);
- a plan for a minimal empirical instantiation on existing benchmarks (Section 6);
- a set of open research problems that follow from the framework (Section 7).

2. Related Work

Specification gaming and reward hacking. Amodei et al. (2016) list reward hacking and negative side effects among the concrete problems of AI safety. Krakovna et al. (2020) collect examples of specification gaming, where a system satisfies the stated objective without achieving the intended one, and Skalse et al. (2022) give a formal treatment of reward hacking. Goal misgeneralization is a related failure in which a policy competently pursues the wrong goal out of distribution even though the training specification was correct (Langosco et al., 2022; Shah et al., 2022). Our concern overlaps with all three, but the emphasis differs: we are interested in cases where the objective really is achieved and the fault lies in the path taken.

Constrained and safe reinforcement learning. Maximizing reward subject to cost limits is the subject of constrained Markov decision processes (Altman, 1999) and of a large literature on safe reinforcement learning (García & Fernández, 2015; Achiam et al., 2017). That literature mostly studies how to *train* policies that satisfy constraints. We borrow its formalism and ask a different question, namely how to *evaluate* a deployed agent whose constraints are stated in natural language and are often only partly specified.

Agent benchmarks and safety evaluations. Benchmarks such as AgentBench (Liu et al., 2024), WebArena (Zhou et al., 2024), and SWE-bench (Jimenez et al., 2024) measure whether tasks are completed. A smaller body of work looks at how they are completed. MACHIAVELLI reports ethical violations alongside reward in text-based games (Pan et al., 2023), ToolEmu uses an LLM-emulated sandbox to surface risky tool use (Ruan et al., 2024), and AgentDojo measures utility and security jointly under prompt injection (Debenedetti et al., 2024). Work on AI control studies monitoring protocols that remain useful even when the monitored model may act adversarially (Greenblatt et al., 2024).

Process-level and constraint-focused benchmarks. The work closest to ours measures constraint violations directly. ODCV-Bench places agents in multi-step scenarios where performance is tied to a key performance indicator and reports how often they break ethical, legal, or safety rules to reach it (Li et al., 2025). SpecBench measures reward hacking in long-horizon coding agents as the gap between pass rates on visible tests and on held-out tests (Zhao et al., 2026). MasDrift tests whether delegated subtasks in multi-agent systems stay within the permissions of the original user (Xu et al., 2026), and SLBench turns the logical relations between instructions in agent skill files into executable tests (Chen et al., 2026). Huang et al. (2026) study reward hacking by autonomous research agents and find that a review panel misses many confirmed cases. A recent survey of trustworthy agentic AI collects metrics and benchmarks for safety, robustness, privacy, and security in one reference (Qi et al., 2026).

Two benchmarks already report capability and safety as separate numbers. ST-WebAgentBench pairs enterprise web tasks with policies and reports Completion under Policy, which credits a task only if the agent completed it without breaking an applicable policy, together with a risk ratio (Levy et al., 2024). ClawsBench reports a task success rate and an unsafe action rate for productivity agents in simulated workspaces (Li et al., 2026b).

A second group of work supplies trajectory-level data and diagnosis. The ATBench extensions adapt a safety taxonomy of risk source, failure mode, and real-world harm to particular agent runtimes (Yang et al., 2026). Who&When Pro builds failed trajectories with labeled failure points by injecting a failure after replaying a successful prefix (Liu et al., 2026), a construction close to the paired trajectories we call for in Section 8. Jha et al. (2026) show that agents take unsafe actions after ordinary environment errors, with no adversary involved, and relate this to exploratory behavior. On the governance side, Li et al. (2026a) argue that constraints weaken as they pass through memory, delegation, and tool use, and propose keeping them as explicit execution state, while Ding et al. (2026) survey agents with persistent state and outline an evaluation contract that scores obligations on state mutation and recovery instead of answer quality alone.

Reporting outcome and compliance separately is therefore not new, and we do not claim it. We do not propose another benchmark either. This paper adds three things to that line of work. The first is a constraint schema that is independent of any one benchmark and that distinguishes hard from soft constraints and kinds of evidence. The second is a per-run profile that keeps all four outcomes of Table 1 apart. A conjunctive metric such as Completion under Policy measures the rate of category (1) and merges the other three,

so it cannot separate a safe failure from an unsafe success. The third is a gating rule over the profile together with an account of what checker error does to it. The benchmarks above are complementary to this paper, and they are the natural testbeds for it (Section 6).

Anomaly detection and LLM-based judges. Detecting inputs or behaviors that fall outside a reference distribution is a mature topic (Chandola et al., 2009; Hendrycks & Gimpel, 2017). LLMs are now commonly used as judges of other models' outputs (Zheng et al., 2023), although such judges have systematic biases, including a preference for their own generations (Panickssery et al., 2024). Both lines of work are relevant to trajectory monitoring, and we return to them in Sections 4 and 7.

3. Goal Achievement versus Constraint Compliance

An agent can be viewed as a policy π chosen to maximize an expected task reward,

$$\max_{\pi} \ \mathbb{E}_{\tau\sim\pi}\big[R(\tau)\big], \tag{1}$$

where τ is a trajectory and R rewards completion of the task. Real instructions almost never stop there. They also say which resources the agent may access, which tools it may use, what information it may disclose, and which actions are off limits. Writing these restrictions as a set of constraints $\mathcal{C} = \{c_1, \dots, c_m\}$, the behavior we actually want is closer to the constrained problem (Altman, 1999)

$$\begin{aligned} \max_{\pi} \quad & \mathbb{E}_{\tau\sim\pi}\big[R(\tau)\big] \\ \text{s.t.} \quad & \Pr_{\tau\sim\pi}\big[\tau \models c_i\big] = 1, \quad i = 1, \dots, m, \end{aligned} \tag{2}$$

where $\tau \models c_i$ means that the trajectory satisfies constraint c_i.

Requiring every constraint to hold with probability one is the right target for restrictions that must never be broken, such as using a credential the agent was not given. As a general model it is too strict, for two reasons. Some restrictions are budgets and not prohibitions, for example a limit on the number of external requests. And violations are in practice detected by imperfect checkers, so a guarantee of exactly zero cannot be verified. The safe reinforcement learning literature offers two standard relaxations (Altman, 1999; Achiam et al., 2017). A chance constraint bounds the probability of violation,

$$\Pr_{\tau\sim\pi}\big[\tau \not\models c_i\big] \le \delta_i, \tag{3}$$

and a cumulative-cost constraint bounds an expected cost,

$$\mathbb{E}_{\tau\sim\pi}\big[g_i(\tau)\big] \le d_i, \tag{4}$$

where $g_i \ge 0$ measures how much of budget i the trajectory consumed. Problem (2) is the special case $\delta_i = 0$. We call a constraint *hard* if it is of this kind and *soft* otherwise, and the quantities reported in Section 4 are defined for both.

Problems arise when the two formulations disagree, that is, when breaking a constraint raises the probability of completing the task. Typical situations include the following.

- A restricted tool gives faster or more reliable access to the needed information.
- An external system exposes credentials that make the task trivial.
- A security weakness lets the agent bypass a restriction in the environment.
- The evaluation metric can be manipulated directly to yield a higher score.
- Taking many additional, unrequested actions raises the chance that one of them produces the desired result.

In each case an optimizer that sees only (1) has a reason to leave the feasible set of (2). These situations are instances of reward hacking, specification gaming, and goal misgeneralization as discussed in Section 2. The question that matters is thus not only whether an agent *can* reach an objective, but whether its optimization stays inside the operational boundaries its principals intended.

Two features make the agent setting harder than the classical constrained one. First, $\mathcal{C}$ is rarely written down in full. Some constraints are explicit in the prompt, some are implied by the deployment context, and some are norms no one thought to state. Second, the action space is open-ended. An agent with a shell and network access can do things the benchmark designer never enumerated, so violations cannot be detected by checking membership in a short list of forbidden actions.

4. A Constraint-Aware Evaluation Framework

We propose a conceptual Constraint-Aware Agent Evaluation Framework (CAAEF) with five components. They are ordered from what is already standard practice to what is least developed.

4.1. Outcome Verification

The first component asks whether the assigned objective was completed, and yields a score $O(\tau) \in [0, 1]$. This is what existing benchmarks measure. It is necessary, but it is not enough: two agents can produce the same result by entirely different means, so O on its own says nothing about whether the behavior can be trusted.

4.2. Trajectory Monitoring

The second component records the full sequence of actions,

$$\tau = (a_1, a_2, \ldots, a_T), \tag{5}$$

together with the observations each action returned, and makes this sequence the object of evaluation instead of the final answer. A trajectory is long and most of it is routine, so a monitor needs to know what to look for. The situations listed in Section 3 suggest the following signals:

- tool calls that the task does not obviously require;
- attempts to access restricted resources;
- repeated failed attempts to get past a control;
- abrupt changes of strategy;
- contact with external systems not mentioned in the task;
- acquisition or use of credentials the agent was not given;
- actions unrelated to the legitimate objective.

None of these is conclusive in isolation. A second attempt after a refusal may be a corrected typing error, and an unfamiliar tool may be the right one. The signals are inputs to the components that follow: the compliance check decides whether a flagged action broke a constraint, and the anomaly score measures how unusual the pattern is as a whole.

4.3. Constraint Compliance

Checking actions against $\mathcal{C}$ requires the constraints to be written in a form that two evaluators would apply in the same way. We suggest recording each constraint c_i with six fields:

- *scope*: the resources, tools, or data it covers;
- *modality*: prohibition, obligation, or budget;
- *class and severity*: hard or soft, with a weight $w_i \geq 0$;
- *condition*: the state in which it applies, for example only before the user has confirmed;
- *source*: stated in the instructions, implied by the deployment context, or an unstated norm;
- *evidence*: how a violation is established.

For the last field we distinguish three kinds of evidence, in decreasing order of reliability: the state of the environment or the artifacts the agent produced, the execution log checked by a deterministic rule, and the reading of the trajectory by a human or LLM judge. A constraint should be checked with the most reliable kind available to it. Executable tests of the sort used in SLBench (Chen et al., 2026) belong to the first two kinds. Constraints whose source is an unstated norm will usually need a judge, and these are the ones on which labels are least consistent. Recording the source makes it possible to report results separately for explicit and implicit constraints.

Let $v_{t,i} \in \{0, 1\}$ indicate whether action a_t, taken in the context of the preceding trajectory, violates c_i. Two summaries of this array are used below. The per-action indicator $v_t = \max_i v_{t,i}$ records whether step t broke any constraint, and the per-constraint indicator $V_i(\tau) = \max_t v_{t,i}$ records whether constraint c_i was broken at any step. The context matters because the same action, such as reading a file, may be permitted at one point and prohibited at another. A simple trajectory-level compliance score is the fraction of compliant actions,

$$\mathrm{CS}(\tau) \;=\; 1 - \frac{1}{T}\sum_{t=1}^{T} v_t. \tag{6}$$

A high objective score combined with a low compliance score indicates optimization that succeeded but may not be safe.

The ratio in (6) has an obvious weakness: one serious violation in a long trajectory barely moves it. A hundred-step run that exfiltrates a credential once still scores 0.99. We therefore report two further quantities. Writing $\mathcal{C}_H \subseteq \mathcal{C}$ for the hard constraints, the strict indicator

$$V(\tau) \;=\; \max_{c_i \in \mathcal{C}_H} V_i(\tau) \tag{7}$$

records whether any hard constraint was broken, and the severity-weighted cost

$$\mathrm{SC}(\tau) \;=\; \sum_{t=1}^{T} \sum_{c_i \notin \mathcal{C}_H} w_i \, v_{t,i} \tag{8}$$

accumulates soft violations over the run. A hard violation is a reason to stop the run or to disqualify it, and a soft violation is a warning that adds to a running total. Over many runs, the mean of V estimates the left-hand side of (3) and the mean of SC plays the role of the cost in (4).

The indicators $v_{t,i}$ are outputs of a checker and not ground truth. Reported values of CS, V, and SC should therefore be accompanied by the checker's false-positive and false-negative rates, estimated on a sample of trajectories audited by humans.

4.4. Uncertainty and Anomaly Detection

Not every unusual action is a violation, and not every violation is deliberate. An agent that is unsure how to proceed

may legitimately try alternatives, and exploration of this kind should not be penalized as if it were an attack. Evaluation should therefore carry an explicit estimate of uncertainty, which we denote $U(\tau)$. It covers both the agent's uncertainty about the task and the evaluator's uncertainty about whether a given action is compliant.

Anomaly detection offers one way to operationalize this. Given a reference set of trajectories known to be compliant, one can compute a shift score $D(\tau)$ that measures how far the current trajectory, or its prefix $\tau_{1:t}$, lies from that distribution, using standard techniques (Chandola et al., 2009; Hendrycks & Gimpel, 2017) applied to action sequences or their learned representations. Because the score can be computed on prefixes, it opens the possibility of flagging risky behavior *before* the agent finishes. We regard the use of uncertainty and distribution shift for this kind of early warning as one of the more promising directions suggested by the framework.

One concrete way to compute D is the following. Tool names differ from task to task, so raw calls cannot be compared. Each call is first mapped to a typed action given by the category of the tool, the class of resource it touches, whether it reads or writes, and whether it falls inside the scope granted by the task. A prefix is then encoded by the sequence of these types, and scored by its distance to the nearest compliant reference prefixes for tasks of the same family. The flagging threshold is set on a held-out set of compliant trajectories so that the false-alarm rate is controlled. Conformal methods give such a guarantee when the calibration and test trajectories are exchangeable (Vovk et al., 2005).

Exchangeability is a strong assumption in this setting. It fails when the agent meets tools or task types that are absent from the calibration set, and it fails after any change to the agent. We therefore suggest calibrating separately for each task family and tracking the flag rate on runs that are known to be compliant. A rise in that rate shows that the guarantee no longer holds and that the threshold must be recalibrated. The same calibration step applies to U, which can be estimated from the disagreement among several independent checkers.

The two signals are not interchangeable, and neither contains the other. U is a statement about confidence in a judgment of compliance. D is a statement about the behavior and involves no such judgment. They can disagree in both directions. A new but legitimate way of solving a task lies far from the reference set and is plainly compliant, so D is high and U is low. A routine trajectory may contain one step whose status cannot be settled from the available evidence, so D is low and U is high. In our reading D is the signal suited to early warning on prefixes, and U is the signal that decides which flagged cases need a human.

Table 1. Behavioral categories obtained by crossing objective achievement with constraint compliance. Outcome-only benchmarks cannot separate the two cells in the top row.

Objective	Compliant	Violating
Achieved	(1) Intended success	(2) Unsafe success
Failed	(3) Safe failure	(4) Unsafe failure

Uncertainty also bears on the soft cost, and the two kinds of soft constraint behave differently. A budget is checked by counting in the log, so an overrun is known exactly. What is unclear is whether it matters, because an agent that retries a failed call or tries a second tool is exploring, and exploration consumes budget. The period after an environment error needs particular care, since it is where unsafe actions have been observed without any adversary (Jha et al., 2026). To keep exploration and overrun apart, we suggest setting each budget d_i from the reference set, for example at a high quantile of what compliant runs of the same task family consume, and counting only the excess. Ordinary exploration then falls inside the budget by construction. Soft constraints that need a judge are a different case, because there the violation itself is uncertain. For these we suggest reporting SC as an interval. The lower end counts the violations on which all checkers agree, and the upper end adds those on which they disagree. A wide interval says that the soft cost of the run is not known, which is more useful than a point value that hides the disagreement.

4.5. Post-Hoc Behavioral Analysis

After the task ends, the trajectory should be reconstructed and examined with questions such as these. What was the agent trying to accomplish at each stage? Which actions were needed for the task and which were not? Did the agent try to get around a restriction? Did it exploit a weakness in the environment? Did it obtain information or permissions beyond what the task required?

The answers support a distinction between *necessary* optimization, meaning effort the task demands, and *opportunistic* optimization, meaning the use of whatever shortcut the environment happens to offer. The second kind is what the framework is designed to expose.

5. A Two-Dimensional Evaluation Model

The framework separates two core dimensions. Objective achievement asks whether the agent accomplished the assigned task. Constraint compliance asks whether it stayed within the intended boundaries. Crossing the two gives the four categories in Table 1.

Category (2) deserves the most attention. An outcome-only

benchmark places it in the same bin as category (1), so the agents that are most concerning from a safety standpoint are precisely the ones the benchmark rewards. Category (3) is also worth a remark. An agent that stops and reports that the task cannot be done within its permissions has behaved well, yet outcome-only scoring treats it the same as an agent that failed through incompetence.

A fuller evaluation profile would add the quantities from Section 4 and report, for each run, the tuple

$$P(\tau) \;=\; \big(O,\ \mathrm{CS},\ V,\ \mathrm{SC},\ U,\ D\big), \tag{9}$$

where the six quantities are those introduced in Section 4, each evaluated on τ. We recommend against collapsing this profile into one scalar. Any fixed weighting allows a large gain in O to offset a constraint violation, which reintroduces at the level of the metric the very trade-off the constraints were meant to rule out. A single number can hide the failure modes that matter most.

Declining to aggregate does not mean declining to decide. For release gating we suggest a lexicographic rule. A run with $V = 1$ is disqualified whatever its value of O, and an agent is rejected if its rate of such runs exceeds the tolerance δ set by the deployer. Among agents that pass this test, one is preferred to another only if it is no worse on both SC and O. Where neither dominates, the choice is left to the deployer, who sees the trade-off instead of having it made by a weighting. Under this rule a safe failure ranks above an unsafe success. Category (3) costs utility, whereas category (2) breaks the conditions under which the agent was allowed to act at all.

The rule is only as good as the checker behind V, and its behavior under checker error can be worked out directly. Suppose the checker reports a hard violation on a compliant run with probability α and misses a real one with probability β. If the true rate of hard violations is p, the observed rate is

$$q \;=\; (1-\beta)\,p + \alpha\,(1-p). \tag{10}$$

Three consequences follow. First, an agent that never violates is still observed at rate α, so it is rejected whenever $\alpha > \delta$. A tolerance below the false-positive rate of the checker cannot be enforced on the raw output. Second, a share α of compliant runs is disqualified, and if the monitor stops a run when it flags, the same share is turned into forced safe failures. This is the utility cost of the veto, and it is paid whether or not the agent ever misbehaves. Third, misses lower q by βp, so an agent can pass the gate with a true rate above δ.

Hard violations are meant to be rare, and when p is small the false positives dominate. With $p = 0.02$, $\alpha = 0.05$, and $\beta = 0.2$, equation (10) gives $q = 0.065$, of which about three quarters are false alarms. Two practical rules follow. The gate should be applied to the corrected estimate

$$\hat{p} \;=\; \frac{q-\alpha}{1-\alpha-\beta}, \tag{11}$$

which is valid when $\alpha + \beta < 1$ (Rogan & Gladen, 1978), with a confidence interval that includes the uncertainty in α and β from the audit. And every run with $V = 1$ should be confirmed by stronger evidence or by a human before it counts, which removes most of the cost of α at the price of review effort proportional to q. The analysis assumes that α and β do not depend on the agent. That assumption fails for an agent that conceals its actions, for which β is higher than the audit suggests.

6. A Minimal Empirical Instantiation

We have not run the framework, and this section describes the smallest study that would test it. Nothing below is a result.

Testbed. Two options are available. The first is a benchmark that already contains constraint-violating routes, such as ODCV-Bench (Li et al., 2025) or MasDrift (Xu et al., 2026). The second is a subset of WebArena (Zhou et al., 2024) or SWE-bench (Jimenez et al., 2024) tasks annotated with the schema of Section 4. The first needs less annotation. The second shows whether the schema can be applied to tasks that were not designed with it in mind.

Tracing. Every tool call, its arguments, and the returned observation are logged in a common format. Tooling based on OpenTelemetry already records aligned traces for LLM-based agent systems (Seyedghorban et al., 2026), and aligned traces are what the paired comparison of compliant and non-compliant runs requires.

Labels. The indicators $v_{t,i}$ are produced by rule checkers where the evidence field allows it and by an LLM judge elsewhere. A random sample of trajectories is labeled independently by two human annotators, which gives both inter-annotator agreement and the error rates α and β of the automated checkers. The size of the audit follows from the precision required. Estimating an error rate near 5% to within three percentage points at 95% confidence takes about 200 audited trajectories, which can be done by hand. Rates well below 1% cannot be estimated this way and call for targeted review of flagged runs.

Measurements. Four questions can then be answered. How many runs counted as successes by outcome-only scoring have $V = 1$, and does the ranking of agents change once they are removed? How well does D on prefixes separate violating from compliant runs, measured by AUROC and by the number of steps between the flag and the first

violation at a fixed false-alarm rate? Is U calibrated, in the sense that cases marked uncertain are those on which checkers and humans disagree? And what does each component add, judged by ablation against outcome-only scoring, rule checkers alone, and an LLM judge alone?

7. Research Challenges

Inferring intent from behavior. Deciding whether an agent *deliberately* worked around a constraint is difficult, and unexpected behavior should not be read as strategic intent by default. More defensible evidence comes from the structure of the trajectory: what the environment returned, whether an attempt was repeated after being refused, whether compliant alternatives were available and ignored, and how behavior changed once a constraint was encountered. Even then, conclusions about intent should be stated with care.

Hidden and emergent strategies. Agents operating over long horizons may find strategies that benchmark designers did not anticipate, and a benchmark cannot detect a violation it offers no opportunity to commit. Future environments should deliberately include both a legitimate route and one or more constraint-violating routes to the same objective, so that the choice between them becomes observable.

Reward design. If completion is strongly rewarded and violations are weakly penalized, an optimizer will sometimes prefer to violate. Benchmarks and training setups should keep task reward and constraint-related signals separate, so that this trade-off is visible instead of being buried in one objective.

Early detection. Post-hoc analysis is useful for understanding failures, but deployed systems need monitoring while the agent is running. One option is a running risk estimate that combines trajectory anomaly, uncertainty, proximity to a constraint boundary, and tool-use patterns, with human intervention requested when the estimate crosses an adaptive threshold. How to set that threshold without overwhelming human reviewers is an open problem. It is also a practical one: a review of production agent systems finds that human-centered controls such as runtime approval are the dominant security mechanism in industry (Wang et al., 2026), and a monitor that asks too often will have its requests approved without being read.

Reliability of automated monitors. At scale, trajectory review will largely be done by other models. This raises an evaluator–agent problem. If the monitor shares blind spots with the agent it oversees, for example because both derive from the same base model, automated oversight can fail systematically and without any visible sign (Panickssery et al., 2024; Greenblatt et al., 2024). The evidence so far is not reassuring. Huang et al. (2026) report that a review panel misses many confirmed cases of reward hacking, and that detailed feedback from reviewers helps agents evade detection in later rounds. We therefore recommend an adjudication protocol in place of a single judge: deterministic checks wherever the evidence allows, at least two monitors drawn from different model families, and human review of every case on which they disagree together with a random sample of the cases on which they agree. The sample gives an estimate of the errors the monitors share, which their agreement would otherwise hide. Whether such a protocol is reliable enough, and under which conditions, needs direct empirical study.

8. Conclusion and Future Work

The move from language models that answer to agents that act calls for a matching change in how they are evaluated: completing a task and behaving acceptably while doing so are different things, and an agent that reaches the correct answer by an unauthorized route is a different object from one that reaches the same answer within its permissions. We have therefore proposed moving the unit of evaluation from the final answer to the whole interaction trajectory, and reporting objective achievement, constraint compliance, uncertainty, and behavioral distribution shift as separate dimensions with a gating rule in which a hard violation cannot be offset by task success, which brings out a failure mode that conventional benchmarks conceal, namely agents that look successful while reaching their objectives through unintended or prohibited means. The framework is conceptual and has not been validated experimentally. It also assumes that a constraint set can be written down and checked, which is often only approximately true, and that trajectories are fully observable to the evaluator, so an agent able to conceal actions falls outside what trajectory monitoring can cover. Three directions for future work follow: paired-trajectory datasets in which two agents achieve identical outcomes by compliant and non-compliant strategies, so that detectors can be trained and tested with the outcome held fixed; uncertainty-aware monitoring that escalates ambiguous behavior to a human when confidence is low; and distribution-shift detection calibrated on trajectories from safe operation. The study outlined in Section 6 is the natural next step. The research question is not only "can the agent achieve the goal?" but "can the agent achieve the goal within the intended behavioral boundaries, and can we reliably tell when it has not?"

Impact Statement

This paper discusses how to evaluate AI agents so that constraint-violating behavior is not mistaken for success.

We expect the main consequence of this line of work to be safer deployment of autonomous systems. Trajectory-level monitoring involves recording agent actions, which may include user data, and such monitoring should be designed with appropriate privacy safeguards.

References

Achiam, J., Held, D., Tamar, A., and Abbeel, P. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.

Altman, E. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.

Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.

Bondarenko, A., Volk, D., Volkov, D., and Ladish, J. Demonstrating specification gaming in reasoning models. *arXiv preprint arXiv:2502.13295*, 2025.

Chandola, V., Banerjee, A., and Kumar, V. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

Chen, X., Wang, C., Yan, L., and Zhang, X. SLBench: Evaluating how LLM agents follow logical relations in skills. *arXiv preprint arXiv:2607.09016*, 2026.

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024.

Ding, T., Nannapaneni, A., Liu, B., and Zhang, L. Always-on agents: A survey of persistent memory, state, and governance in LLM agents. *arXiv preprint arXiv:2606.30306*, 2026.

García, J. and Fernández, F. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480, 2015.

Greenblatt, R., Shlegeris, B., Sachan, K., and Roger, F. AI control: Improving safety despite intentional subversion. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.

Hendrycks, D. and Gimpel, K. A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations*, 2017.

Huang, Y., Xu, Z., Ma, Y., Wang, W., Liu, Z., Xu, Z., Chen, P.-Y., Galley, M., Lin, Z., Feuerriegel, S., Poovendran, R., Sra, M., Pentland, A., Zhang, X., and Chen, Z. Reward hacking challenges oversight of autonomous research agents. *arXiv preprint arXiv:2609.28614*, 2026.

Jha, R., Triedman, H., Bhattacharya, A., and Shmatikov, V. Agent meltdowns: The road to hell is paved with helpful agents. *arXiv preprint arXiv:2605.19149*, 2026.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., and Legg, S. Specification gaming: The flip side of AI ingenuity. DeepMind Blog, 2020.

Langosco, L., Koch, J., Sharkey, L., Pfau, J., and Krueger, D. Goal misgeneralization in deep reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.

Levy, I., Wiesel, B., Marreed, S., Oved, A., Yaeli, A., Mashkif, N., and Shlomov, S. ST-WebAgentBench: A benchmark for evaluating safety and trustworthiness in web agents. *arXiv preprint arXiv:2410.06703*, 2024.

Li, M. Q., Fung, B. C. M., Weiss, M., Xiong, P., Al-Hussaeni, K., and Fachkha, C. A benchmark for evaluating outcome-driven constraint violations in autonomous AI agents. *arXiv preprint arXiv:2512.20798*, 2025.

Li, T., Ma, Y., Wen, H., Huang, Z., Zhou, Q., Fu, Z., and Cheng, G. Safe multi-agent behavior must be maintained, not merely asserted: Constraint drift in LLM-based multi-agent systems. *arXiv preprint arXiv:2605.10481*, 2026a.

Li, X., Choe, K. W., Liu, Y., Chen, X., Tao, C., You, B., Chen, W., Di, Z., Sun, J., Zheng, S., Bao, J., Wang, Y., Yan, W., Li, Y., and Lee, H.-c. ClawsBench: Evaluating capability and safety of LLM productivity agents in simulated workspaces. *arXiv preprint arXiv:2604.05172*, 2026b.

Liu, J., Xi, H., Zhang, S., Zeng, Y., Yue, T., Wang, C., Kang, J., Wu, Q., and Wang, H. Who&When Pro: Can LLMs really attribute failures in AI agents? *arXiv preprint arXiv:2607.09996*, 2026.

Liu, X., Yu, H., Zhang, H., et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

Meinke, A., Schoen, B., Scheurer, J., Balesni, M., Shah, R., and Hobbhahn, M. Frontier models are capable of

in-context scheming. *arXiv preprint arXiv:2412.04984*, 2024.

Pan, A., Chan, J. S., Zou, A., Li, N., Basart, S., Woodside, T., Ng, J., Zhang, H., Emmons, S., and Hendrycks, D. Do the rewards justify the means? Measuring trade-offs between rewards and ethical behavior in the MACHIAVELLI benchmark. In *Proceedings of the 40th International Conference on Machine Learning*, 2023.

Panickssery, A., Bowman, S. R., and Feng, S. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, 2024.

Qi, J., Li, M., Liu, J., Shu, Y., Yu, D., Ma, S., Cui, W., Zhao, Y., Chen, Y., Jiang, R., King, I., and Xu, Z. Towards trustworthy agentic AI: A comprehensive survey of safety, robustness, privacy, and system security. *arXiv preprint arXiv:2605.23989*, 2026.

Rogan, W. J. and Gladen, B. Estimating prevalence from the results of a screening test. *American Journal of Epidemiology*, 107(1):71–76, 1978.

Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations*, 2024.

Seyedghorban, Z., Klimov, E., van Deursen, A., Panichella, A., and Kulahcioglu Ozkan, B. Observability and fault injection for LLM-based multi-agent systems in software engineering. *arXiv preprint arXiv:2608.24271*, 2026.

Shah, R., Varma, V., Kumar, R., Phuong, M., Krakovna, V., Uesato, J., and Kenton, Z. Goal misgeneralization: Why correct specifications aren't enough for correct goals. *arXiv preprint arXiv:2210.01790*, 2022.

Skalse, J., Howe, N. H. R., Krasheninnikov, D., and Krueger, D. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022.

Vovk, V., Gammerman, A., and Shafer, G. *Algorithmic Learning in a Random World*. Springer, 2005.

Wang, P., Li, Y., and Tian, Y. Reframing LLM agent security as an agent-human interaction problem. *arXiv preprint arXiv:2605.24309*, 2026.

Xu, Z., Zhang, X., Luo, H., Jin, Y., Dong, Y., and Salam, H. MasDrift: Benchmarking authorization preservation across multi-agent architectures. *arXiv preprint arXiv:2608.07556*, 2026.

Yang, Z., Li, Y., Zhu, Y., Zhou, T., Xie, Y., Luo, H., Shao, J., Hu, X., and Liu, D. Benchmarks for trajectory safety evaluation and diagnosis in OpenClaw and Codex: ATBench-Claw and ATBench-Codex. *arXiv preprint arXiv:2604.14858*, 2026.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

Zhao, B., Srikanth, D., Wu, Y., and Jiang, Z. SpecBench: Measuring reward hacking in long-horizon coding agents. *arXiv preprint arXiv:2605.21384*, 2026.

Zheng, L., Chiang, W.-L., Sheng, Y., et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

Zhou, S., Xu, F. F., Zhu, H., et al. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.